



Université de
Technologie de
Compiègne

LO17/AI31 : Indexation et Recherche d'Information

Rapport de Projet DM

Membre du binôme 1 :
Caio ARTUS (Contribution en pourcentage)

Membre du binôme 2 :
Clothilde MARKO-LAFON (Contribution en pourcentage)

Groupe :
LO17, TD jeudi-10h.



Sommaire

1	Préparation du corpus	3
1.1	Présentation des fichiers	3
1.2	Extraction des champs à partir du HTML	3
1.3	Construction du fichier XML et de l'index	4
2	Anti-dictionnaire	5
2.1	Choix de l'unité documentaire	5
2.2	Calcul du TF-IDF	5
2.3	Détermination de l'anti-dictionnaire	5
2.4	Génération du corpus filtré	5
3	Lemmatisation et indexation du corpus	6
3.1	Lemmatisation du corpus filtré	6
3.2	Affinage de l'anti-dictionnaire	6
3.3	Construction des fichiers inverses	6
4	Correcteur orthographique	8
4.1	Pipeline de correction	8
4.2	Génération des candidats par recherche par préfixe	8
4.3	Départage par distance de Levenshtein	8
4.4	Vulnérabilité de l'approche et alternative	8



Introduction

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.



Partie 1

Préparation du corpus

L'objectif de ce premier TD est de construire un fichier XML unique et structuré à partir des fichiers HTML bruts de l'archive ADIT, afin de disposer d'une base propre et homogène pour toutes les étapes d'indexation qui suivront.

1.1 Présentation des fichiers

L'archive est constituée de bulletins électroniques publiés entre 2011 et 2014. Chaque bulletin est un fichier HTML contenant un ou plusieurs articles, répartis dans des rubriques telles que *Focus*, *Événement*, *Actualité Innovation* ou *En direct des laboratoires*. On souhaite en extraire les champs suivants : le numéro de l'article et du bulletin, la date de parution, la rubrique, le titre, le texte, l'auteur, les images avec leur URL et leur légende, et les informations de contact.

1.2 Extraction des champs à partir du HTML

1.2.1 Architecture du parseur

On structure le parseur autour de deux classes aux responsabilités distinctes. `BulletinParser` prend en charge un unique fichier HTML : il l'ouvre, en extrait tous les champs, et produit le fragment XML correspondant. `CorpusParser` orchestre l'ensemble : il itère sur tous les fichiers du répertoire, instancie un `BulletinParser` par fichier, puis assemble les fragments pour produire le fichier `clean_corpus.xml` final.

1.2.2 Double parsing du HTML

On commence par passer chaque fichier dans `BeautifulSoup` pour normaliser le HTML potentiellement malformé, puis on transmet le résultat à `etree.HTML()` de `lxml` pour obtenir un DOM interrogeable en XPath. Cette combinaison tire parti de la robustesse de `BeautifulSoup` pour la correction du HTML et de l'expressivité de XPath pour l'extraction des champs.

1.2.3 Extraction par chemins XPath

Tous les champs sont ciblés par des chemins XPath identifiés en inspectant la structure HTML depuis le navigateur. On applique `strip()` sur les valeurs extraites pour nettoyer les espaces parasites, et `replace()` pour isoler certaines informations encodées dans le texte, comme le numéro de bulletin toujours précédé de la chaîne "BE France".

Extraction du texte. Le texte de l'article partage la même cellule HTML que les images. Pour n'extraire que le contenu éditorial, on utilise un prédicat XPath qui exclut les blocs contenant des images (`not(ancestor::div[img])`) et les spans de légende (`style88`).

Extraction des images. Les images sont ciblées via `div[img]`, l'URL récupérée via l'attribut `src`, et la légende via `following-sibling::span`.

Extraction de l'auteur. Le bloc auteur suit la forme Organisation - Nom - Email : adresse. Une expression régulière souple avec `re.IGNORECASE` en extrait les trois sous-champs. En cas d'échec, les champs sont mis à `None` plutôt que de lever une exception, afin de ne pas interrompre le traitement des autres articles.

Contact. La structure du bloc contact étant trop hétérogène, on l'inclut tel quel dans la balise `<contact>` du XML.



1.3 Construction du fichier XML et de l'index

1.3.1 Formation du fichier XML

Chaque `BulletinParser` produit un fragment XML que `CorpusParser` accroche à la racine `<corpus>`. Chaque article est encapsulé dans une balise `<document>`, conformément à la structure imposée par le TD. L'ensemble est consolidé dans le fichier `clean_corpus.xml`.

1.3.2 Tokenisation

La tokenisation est assurée par `CorpusTokenizer`. On segmente uniquement sur les espaces et les apostrophes, ce qui **conserve le tiret** à l'intérieur des tokens (bien-être reste un token unique, préservant ainsi les termes composés). Chaque token est mis en minuscules, et tous les caractères spéciaux sont supprimés à l'exception de `\w` et du tiret. Les chiffres sont conservés pour permettre la détection des entités numériques au TD4. Enfin, le titre et le texte sont **concaténés avant tokenisation**, de sorte que le titre contribue à la fréquence globale des tokens pour le calcul du TF-IDF.

1.3.3 Construction de l'index inversé

Les fichiers inverses sont construits par la classe `Index` depuis le script `make_index.py`. La méthode `build()` distingue les **champs tokenisés** (titre, texte), découpés par `CorpusTokenizer` et indexés avec leur fréquence et la liste des documents associés, des **champs bruts** (rubrique, bulletin, date, auteur, contact), mis en minuscules et indexés tels quels pour permettre une correspondance exacte. La présence d'images est indexée via le token "image" dès qu'une balise `<images>` est détectée. Les résultats sont sauvegardés en TSV, un fichier par section, ainsi qu'un fichier `lemmes_corpus.csv` consolidant l'ensemble des tokens uniques.

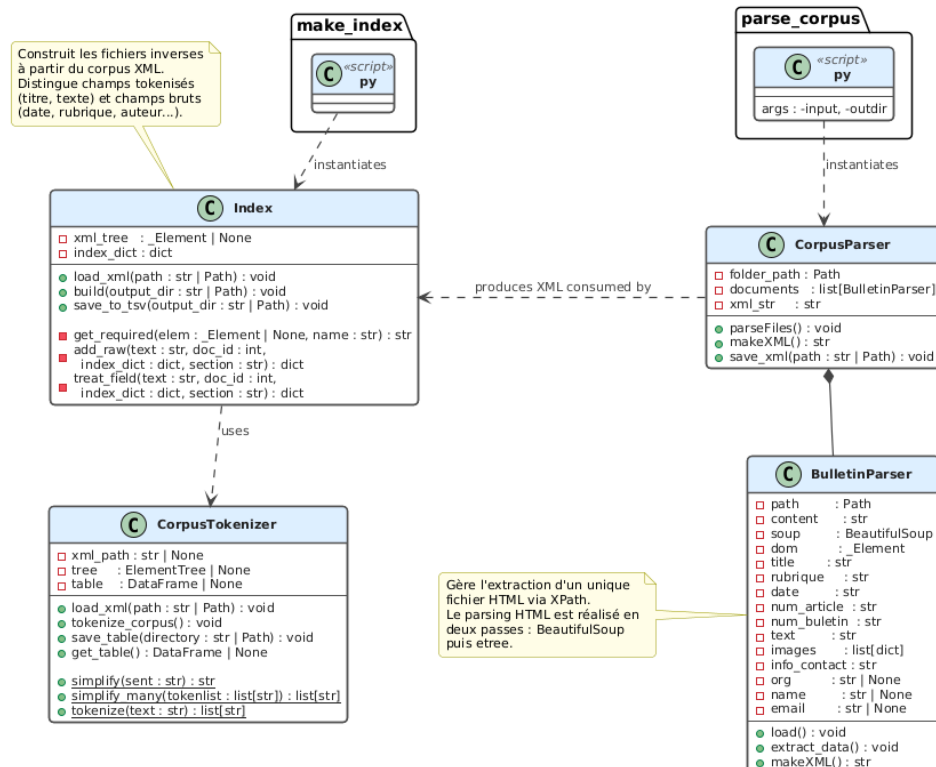


FIGURE 1.1 – UML Indexation (TD1)



Partie 2

Anti-dictionnaire

L'objectif de ce TD est de construire un anti-dictionnaire — une liste de mots non porteurs de sens à exclure du corpus avant indexation — en s'appuyant sur le coefficient TF-IDF pour les identifier automatiquement.

2.1 Choix de l'unité documentaire

On choisit l'**article** comme unité documentaire plutôt que le bulletin. Un bulletin regroupe des articles de rubriques très différentes, ce qui diluerait le signal TF-IDF en mélangeant des contextes hétérogènes. L'article, unité cohérente traitant d'un seul sujet, rend la fréquence d'un token bien plus informative. Ce choix est cohérent avec l'unité documentaire retenue pour la recherche.

2.2 Calcul du TF-IDF

Le calcul est réalisé par la classe `TFIDFProcessor`. Le TF est la fréquence brute du token dans le document, la fréquence normalisée n'étant pas nécessaire ici puisqu'on cherche à identifier les tokens les plus fréquents globalement. L'IDF est calculé selon la formule vue en cours :

$$\text{idf}_t = \log_{10} \left(\frac{N}{\text{df}_t} \right)$$

Avant de calculer df_t , on déduplique les couples (`document_id`, `token`) pour qu'un token apparaissant plusieurs fois dans un même document ne soit compté qu'une seule fois. On n'applique pas de lissage : un token présent dans tous les documents obtient un IDF de $\log_{10}(1) = 0$, ce qui annule directement son TF-IDF. Le TF-IDF final est le produit $\text{tf}_{t,d} \times \text{idf}_t$. Les trois matrices sont sauvegardées séparément en TSV.

2.3 Détermination de l'anti-dictionnaire

La classe `AntiDict` identifie les stopwords en calculant le TF-IDF **moyen** de chaque token sur l'ensemble des documents où il apparaît. On retient la moyenne plutôt que la médiane ou le maximum : elle capture les tokens systématiquement peu discriminants tout en restant insensible aux pics isolés. Tout token dont la moyenne est inférieure ou égale au seuil empirique `thresh = 0.7` est ajouté à l'anti-dictionnaire, stocké dans un set Python pour des recherches en $O(1)$.

L'anti-dictionnaire est encodé sous forme d'une table de substitution à deux colonnes (`token`, `sub`), où une chaîne vide "" indique la suppression. Ce format correspond directement à celui attendu par `substitue`. La méthode `build_from_file()` permet également de charger un anti-dictionnaire manuel, utile pour affiner la liste au-delà de ce que le seuillage automatique peut détecter.

2.4 Génération du corpus filtré

La table de substitution est appliquée au corpus XML via `substitue`, qui parcourt le texte token par token et supprime les stopwords. Le résultat est un fichier XML filtré qui servira de base à la lemmatisation du TD suivant.



Partie 3

Lemmatisation et indexation du corpus

L'objectif de ce TD est de normaliser le vocabulaire par lemmatisation, d'affiner l'anti-dictionnaire à la lumière de cette normalisation, et de construire les fichiers inverses définitifs.

3.1 Lemmatisation du corpus filtré

3.1.1 Architecture du module

On définit une classe abstraite `Stemmer` exposant une interface commune (`make_table`, `transform`, `transform_tolist`, `save_table`), dont héritent `SpacyStemmer` et `SnowStemmer`. Cette conception les rend interchangeables dans le pipeline sans modifier le code appelant.

3.1.2 Implémentation des deux stemmers

`SpacyStemmer` s'appuie sur le modèle français `fr_core_news_sm`. Plutôt que de soumettre le texte brut à `spaCy`, on lui fournit directement les tokens produits par `CorpusTokenizer` en construisant manuellement un objet `spacy.tokens.Doc`. Cela garantit une tokenisation cohérente avec le reste du pipeline et évite tout désalignement avec l'index.

`SnowStemmer` utilise le `SnowballStemmer` de NLTK configuré pour le français. Chaque token est raciné indépendamment via `model.stem()`, sans contexte grammatical : l'approche est déterministe et rapide, mais aveugle aux nuances morphologiques.

3.1.3 Analyse comparative et choix

Le script `3-make_stemmer_tabs.py` applique les deux stemmers sur `nostopwords_corpus.xml` et sauvegarde les tableaux `token/lemma` pour analyse. Sur notre corpus de **14 344 tokens uniques**, `SpaCy` produit **11 285 lemmes uniques** et `Snowball` **9 046 racines uniques**. La courbe de Pareto confirme que les deux approches suivent un profil similaire, avec environ 20% des lemmes couvrant 80% du corpus, `Snowball` ayant toutefois une distribution plus extrême.

La différence qualitative est plus décisive : `Snowball` réduit *optimisation* et *optimiste* à la même racine *optim*, causant des collisions sémantiques problématiques pour un moteur de recherche. **On retient SpaCy** : on perd en rappel (l'utilisateur doit être plus précis), mais on gagne en précision en évitant ces confusions.

3.2 Affinage de l'anti-dictionnaire

La lemmatisation regroupe plusieurs formes fléchies sous un même lemme, augmentant mécaniquement la fréquence de certains termes jusque-là sous le seuil. Par exemple, *permettre* avait un TF-IDF moyen de 0.92 avant lemmatisation ; après, l'absorption de *permet*, *permettent*, *permis* et *permettait* fait chuter cette moyenne à 0.42. De même, *son* absorbe *sa*, *leur* et *leurs*. En maintenant le même seuil $\text{tfidf} \leq 0.7$, on élimine ces nouveaux termes sans ajuster les hyperparamètres. Le corpus doublement filtré est produit par `3-clean_data.py`.

3.3 Construction des fichiers inverses

La classe `Index` est appliquée au corpus doublement filtré et lemmatisé. On obtient un fichier TSV par champ indexé : `index_titre.tsv`, `index_texte.tsv`, `index_rubrique.tsv`,



index_date.tsv, index_auteur.tsv, index_contact.tsv et index_image.tsv, chacun contenant le lemme, sa fréquence totale et la liste des identifiants d'articles associés.

La structure retenue est un dictionnaire Python offrant des recherches en $O(1)$, tout à fait adapté à la taille du corpus. Pour de plus grands corpus, des alternatives comme les **tables de hachage**, les **arbres binaires de recherche** ($O(\log n)$ garanti) ou les **B-Trees** (standard pour l'indexation sur disque) seraient envisageables.

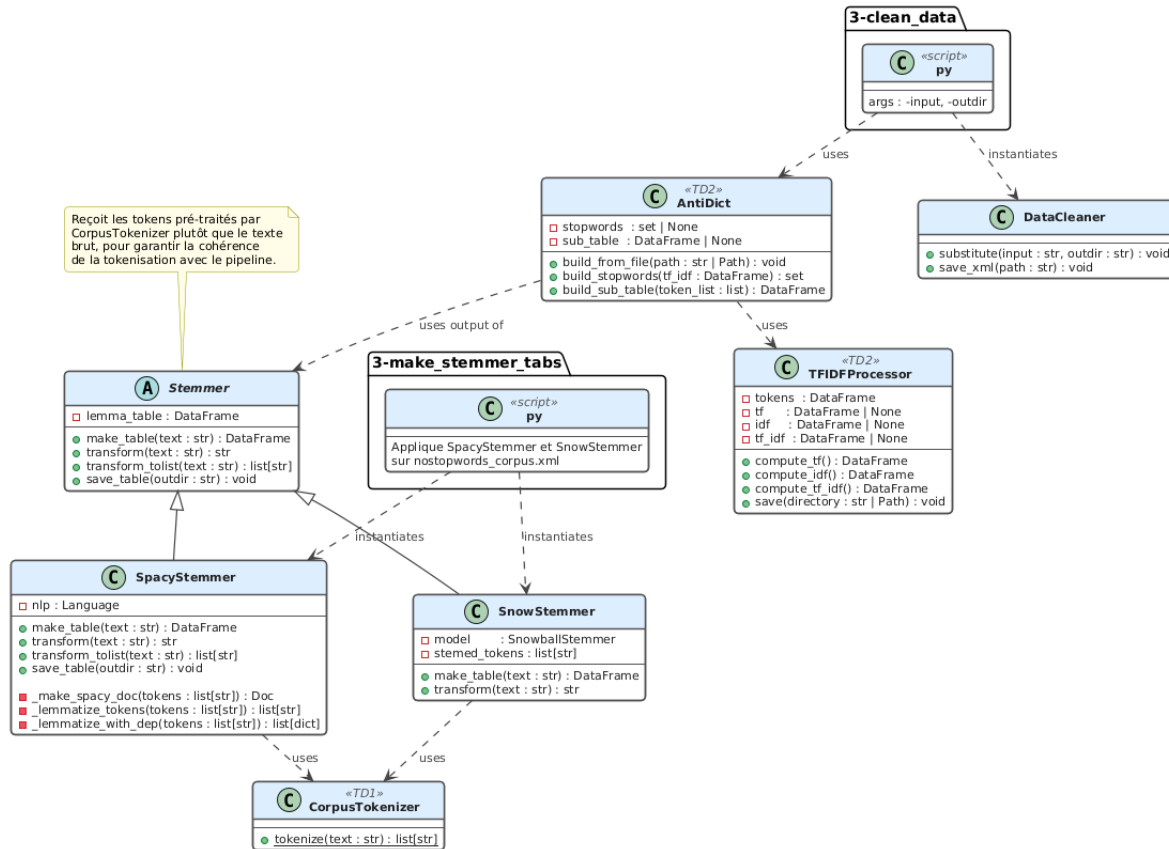


FIGURE 3.1 – UML Stemmer + AntiDict (TD1)



Partie 4

Correcteur orthographique

L'objectif de ce TD est de construire un correcteur orthographique capable, pour chaque token absent du lexique, de proposer le lemme le plus proche. Ce module est intégré dans la classe `Pretraiteur`, qui constitue également le point d'entrée du traitement des requêtes (TD5).

4.1 Pipeline de correction

Le traitement de chaque token suit cinq étapes séquentielles dans `extract_key_words` : tokenisation et lemmatisation via `SpacyStemmer`, filtrage des stop words par comparaison à l'anti-dictionnaire, détection des entités numériques via conversion en float, vérification dans le lexique, et enfin correction orthographique pour les tokens absents.

Le lexique est maintenu sous deux formes : `self.lemmas` (liste ordonnée) pour l'itération dans la recherche par préfixe, et `self.lemma_set` (ensemble) pour les tests d'appartenance en $O(1)$.

4.2 Génération des candidats par recherche par préfixe

La méthode `generate_candidates` réduit l'espace de recherche avant d'appliquer Levenshtein. Elle s'appuie sur trois hyperparamètres fixés empiriquement : `seuilMin = 3` (mots de moins de 3 caractères ignorés des deux côtés), `seuilMax = 4` (différence de longueur maximale tolérée) et `seuilProx = 0.6` (ratio de caractères identiques en position sur la longueur maximale). Un arrêt anticipé est déclenché dès que le taux d'erreur accumulé dépasse $100 - \text{seuilProx}$, ce qui évite de parcourir des mots manifestement trop différents.

4.3 Départage par distance de Levenshtein

`treat_non_existant` applique la logique suivante selon la taille de la liste de candidats : si elle est vide, le token est abandonné ; si elle contient un seul candidat, il est retourné directement ; si elle en contient plusieurs, on retourne celui qui minimise la distance de Levenshtein. Celle-ci est implémentée par programmation dynamique en $O(m \times n)$, avec un coût de substitution binaire. L'approche en deux temps — préfixe puis Levenshtein — limite le calcul quadratique à un petit sous-ensemble de candidats.

4.4 Vulnérabilité de l'approche et alternative

La recherche par préfixe compare les caractères position par position depuis le début du mot. Si la faute porte sur le **début du mot**, le score de proximité sera nul et le bon candidat sera écarté avant même que Levenshtein ne soit appelé. Par exemple, *erecehrche* n'aura aucun caractère commun en position avec *recherche*.

Une alternative robuste consiste à utiliser des **n-grammes de caractères** : on représente chaque mot par ses bigrammes ou trigrammes, puis on calcule une similarité de Jaccard entre les ensembles. Cette méthode est insensible à la position de l'erreur et tolère des transpositions ou insertions en n'importe quel endroit du mot.



Conclusion

Résumé du travail

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Bilan

Décrivez Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Contributions

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.